

A inteligência artificial
**cria códigos
seguros?**





SEGURANÇA

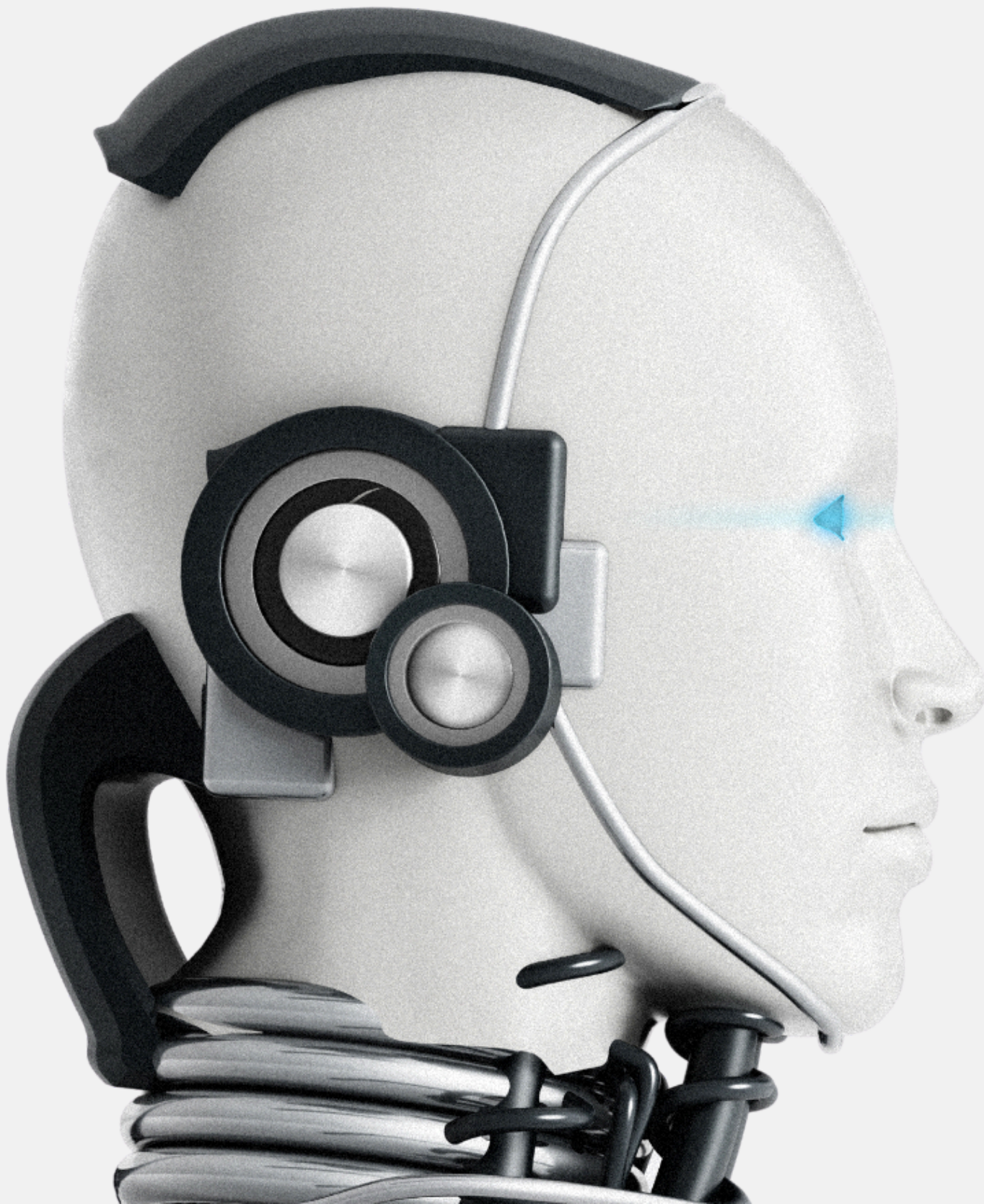
Análise do uso **da inteligência artificial** na criação de softwares mais **inseguros**

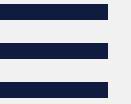
Pergunta Principal:

Usar IA para programar deixa o software mais inseguro?

Perguntas Norteadoras:

- De onde vêm as vulnerabilidades de um software: do código, das dependências ou das pessoas?
- Quando um desenvolvedor confia numa sugestão, o que ele está deixando de verificar?
- Velocidade de produzir código e a segurança do que se produz caminham juntas ou em direções opostas?





De onde vêm as vulnerabilidades de um software:

**do código,
das dependências
ou das pessoas?**

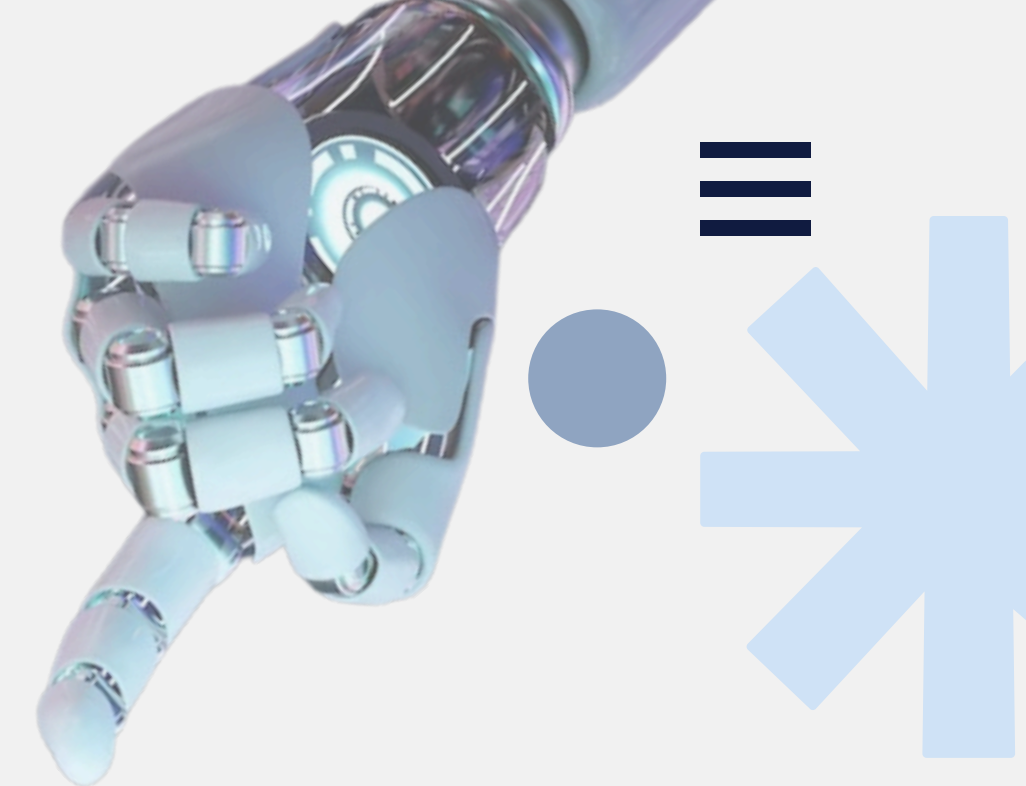


PERI.

SOBRE

CONCLUSÃO

De onde vêm as vulnerabilidades de um software:
do código, das dependências ou das pessoas?



CÓDIGO

40–44%

de falhas em código gerado

(PEARCE, 2022)

18,5%

dos arquivos Python com vulnerabilidades

(MIRHOSEINI, 2025)

DEPENDÊNCIAS

19,7%

de pacotes inventados

205 MIL

pacotes fantasma

(MIRHOSEINI, 2025)

PESSOAS

97%

entregaram falhas de controle de acesso

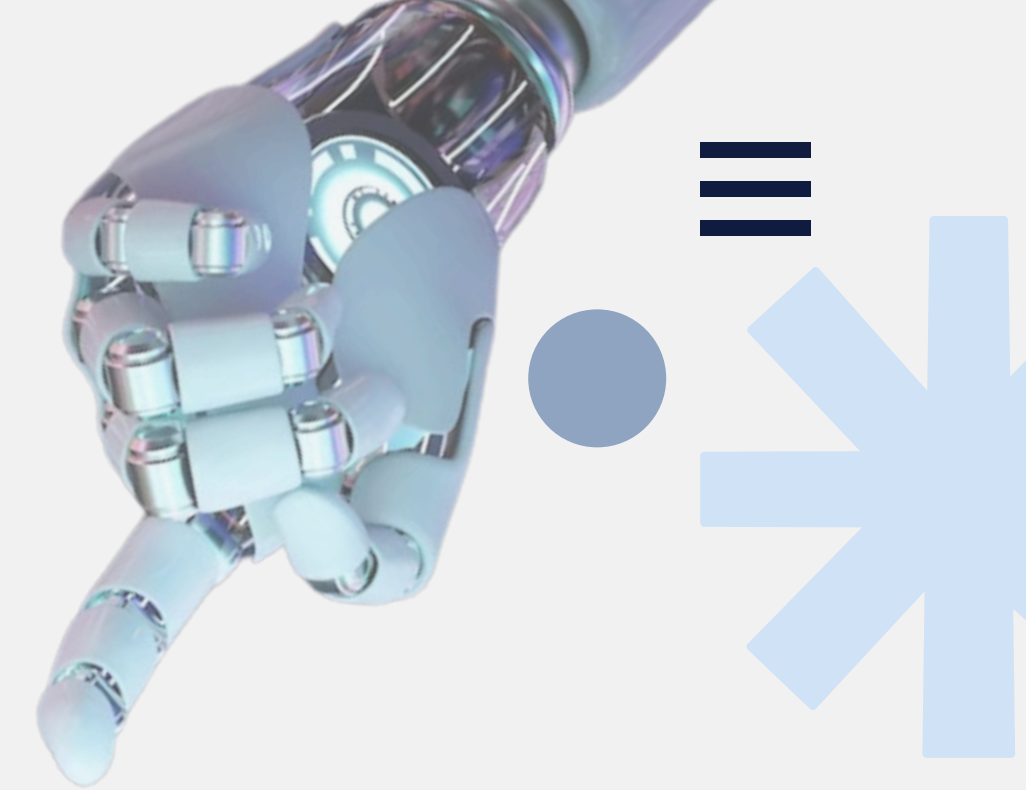
(PERRY, 2023)

PERÍ.

SOBRE

CONCLUSÃO

De onde vêm as vulnerabilidades de um software:
do código, das dependências ou das pessoas?



UMA REAÇÃO EM CADEIA

IA

Reproduz padrão de
código antigo
vulnerável.



ALUCINA

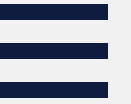
Sugere bibliotecas
inexistentes.



HUMANO

Aceita por excesso de
confiança.

Conclusão: a IA não substitui o erro humano, ela é um **amplificador de risco**.



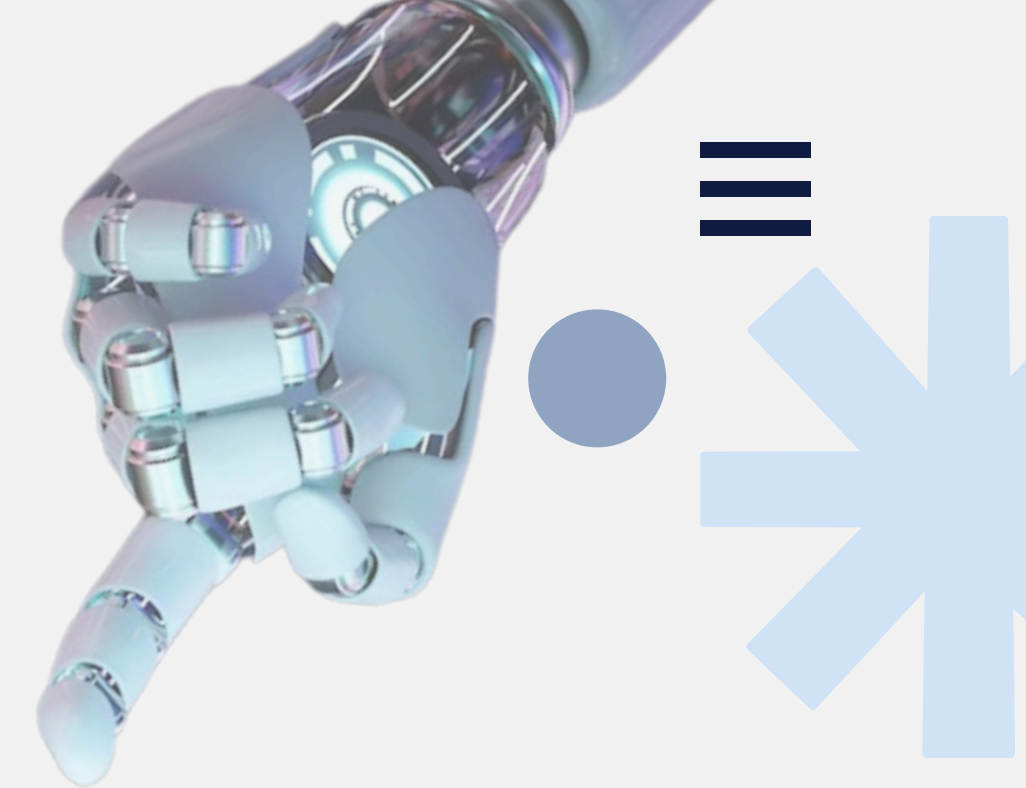
Quando um desenvolvedor confia numa sugestão, o que ele **está deixando de verificar?**



PER2.

SOBRE

CONCLUSÃO



Quando um desenvolvedor confia numa sugestão, o que ele **está deixando de verificar?**

APLICAÇÕES E BANCO DE DADOS

64%

deixaram passar brechas graves de SQL Injection

(PERRY, 2023)

44,4%

das vezes a 1ª sugestão do autocompletar é insegura

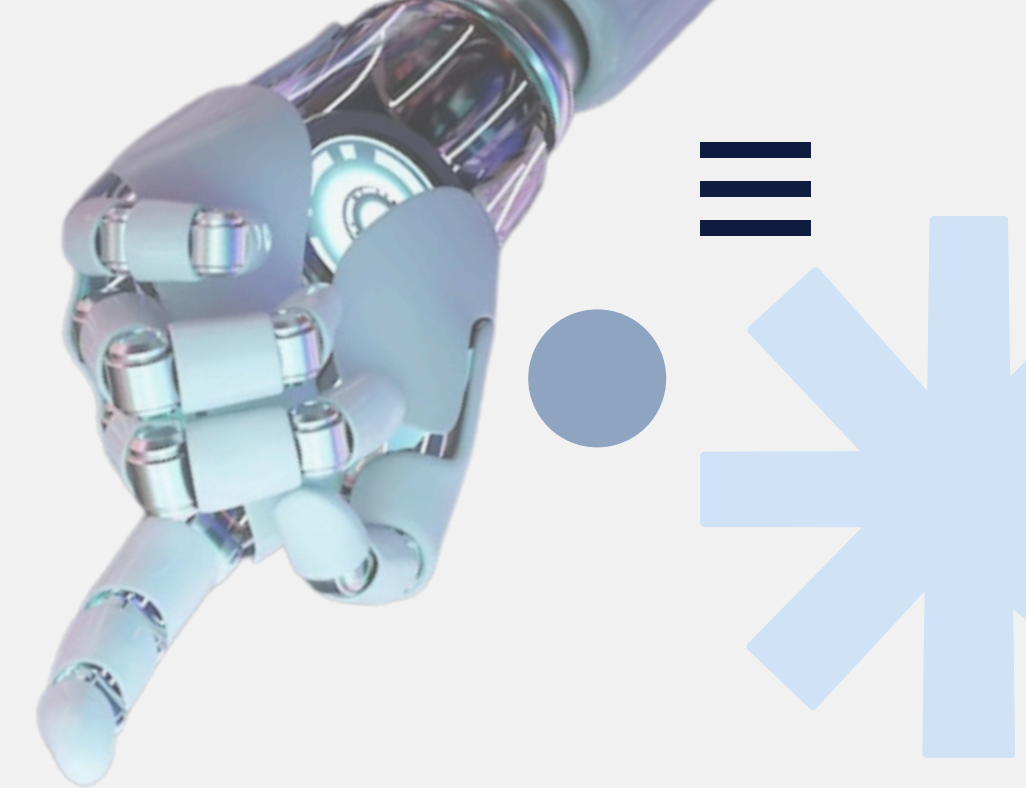
(PEARCE, 2022)

INFRAESTRUTURA E DEPENDÊNCIAS

~50%

dos devs não confirmam se a biblioteca existe

(SPRACKLEN, 2025)



Quando um desenvolvedor confia numa sugestão, o que ele **está deixando de verificar?**

SLOPSQUATTING

58%

das alucinações repetem os mesmos
nomes falsos

(GOODING, 2025)

Vetor de ataque: hackers registram esses
nomes com vírus.

O DEV **VALIDA SE O
CÓDIGO FUNCIONA**, MAS
ABANDONA O PAPEL DE
AUDITOR DE SEGURANÇA.



A velocidade de produzir
código e a segurança do
que se produz

**caminham juntas
ou em
direções opostas?**

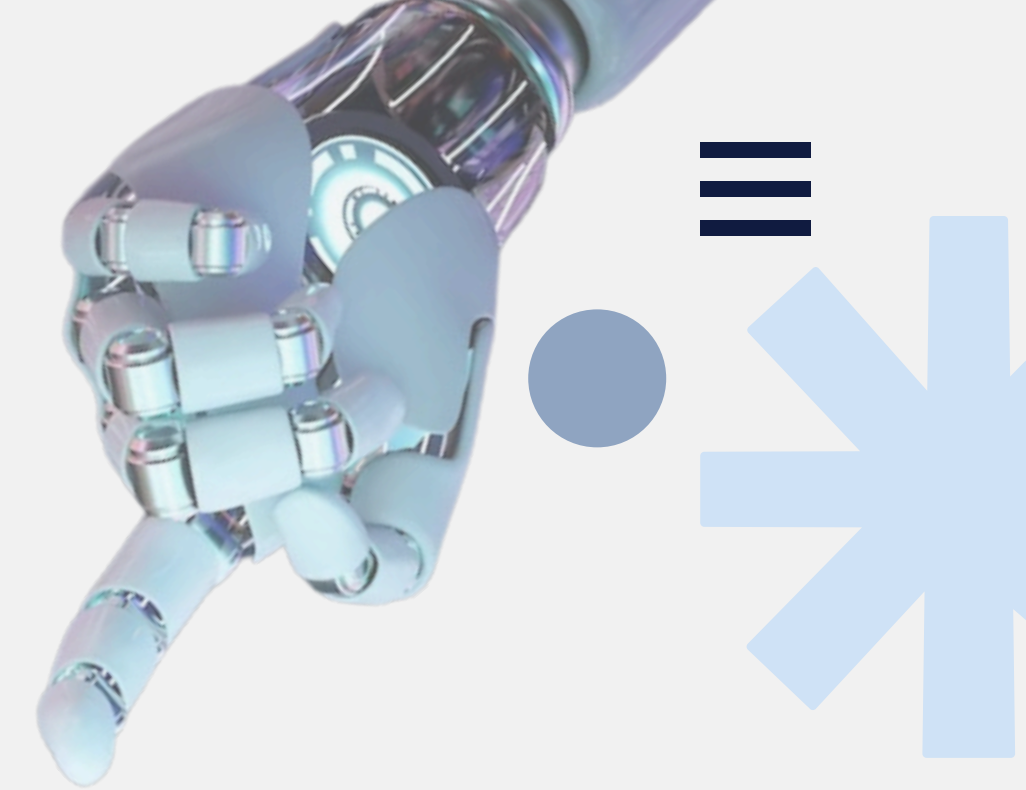


PER3.

SOBRE

CONCLUSÃO

A velocidade de produzir código e a segurança do que se produz
camminham juntas ou em direções opostas?



VELOCIDADE VS. **SEGURANÇA**

PRODUTIVIDADE

83,8%

dos problemas de programação
resolvidos

(AUSTIN, 2021)

QUEDA NA SEGURANÇA REAL

29% → 12%

de códigos 100% seguros

(PERRY, 2023)

O dilema: travar alucinações reduziu a eficiência da IA de 51,4% para 25,3%

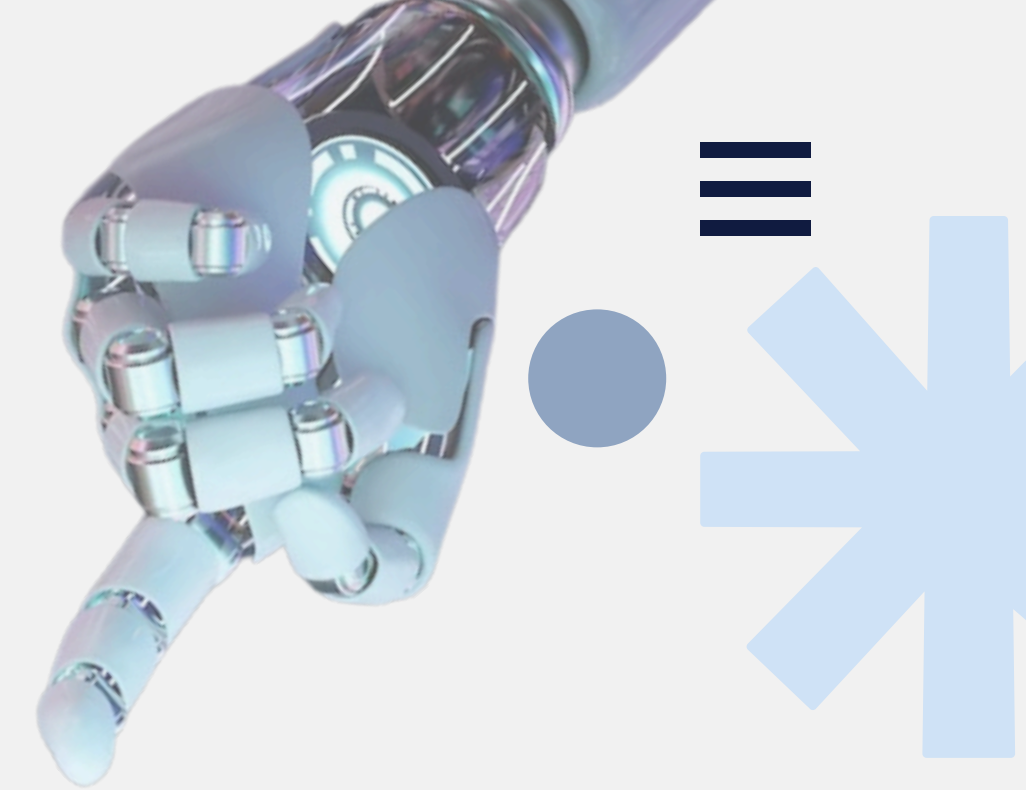
(GOODING, 2025; SPRACKLEN, 2025)

PER3.

SOBRE

CONCLUSÃO

A velocidade de produzir código e a segurança do que se produz
camminham juntas ou em direções opostas?



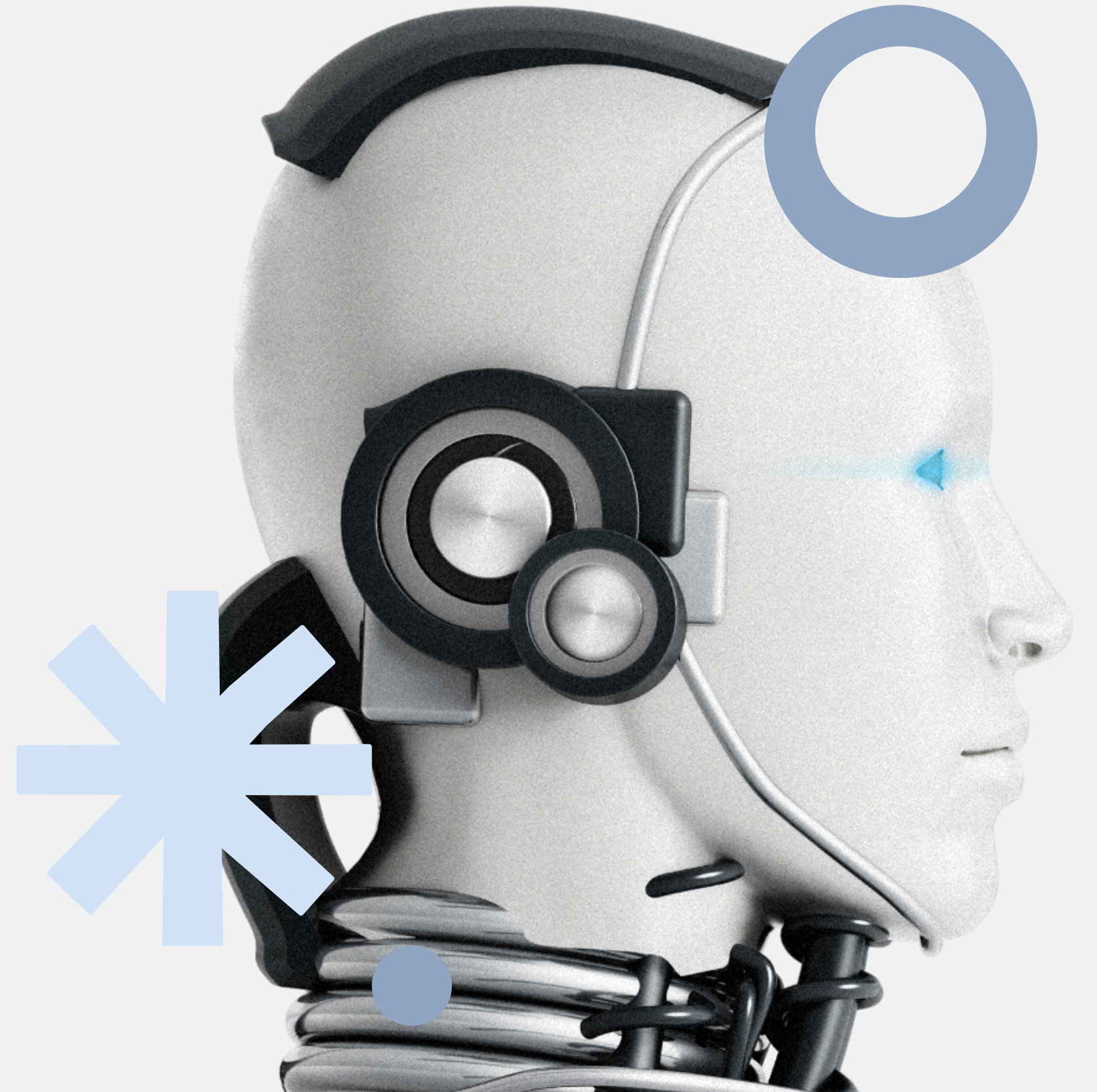
**VELOCIDADE E SEGURANÇA CAMINHAM EM
DIREÇÕES OPOSTAS.**

O **ganho** imediato **de escrita** gera
perda proporcional na **postura**
defensiva.

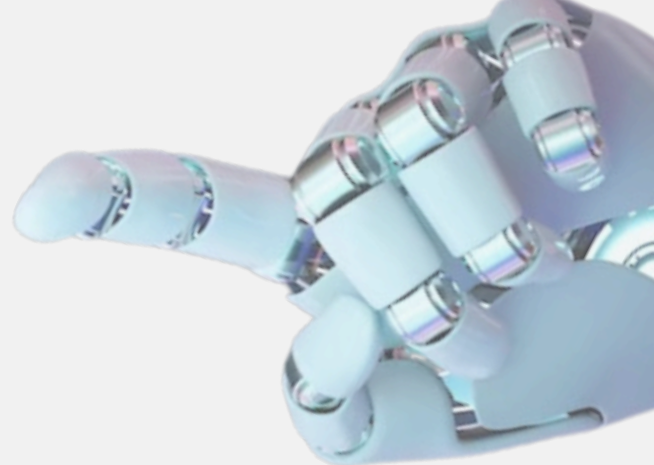
Sem **tempo reinvestido em**
auditoria, a equipe só **acumula**
débito técnico mais **rápido.**



Usar IA para programar
deixa o software mais
inseguro?



CONCLUSÃO



Usar IA para programar deixa o software mais **inseguro**?

SIM. Usar IA para programar deixa o software estatisticamente mais inseguro.

IA

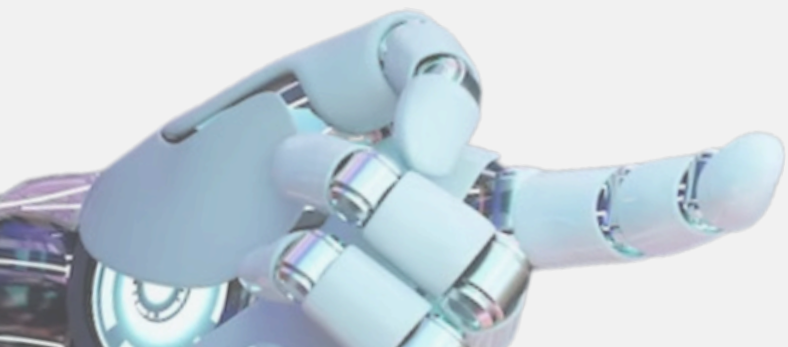
Repete vícios do histórico de treinamento.

HUMANO

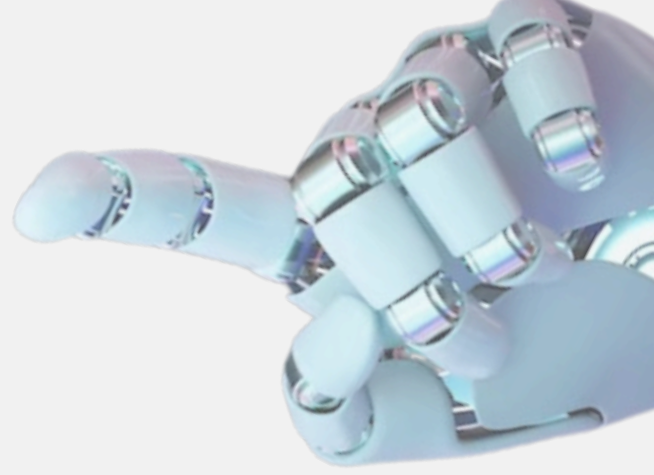
Falsa sensação de competência; menos auditoria.

PROCESSO

Novos vetores de ataque, como pacotes fantasma.



RECOMENDAÇÕES



IA COMO ASSISTENTE. HUMANO COMO AUDITOR CRÍTICO.

1 UTILIZAR

Testes estáticos de segurança (SAST) automatizados.

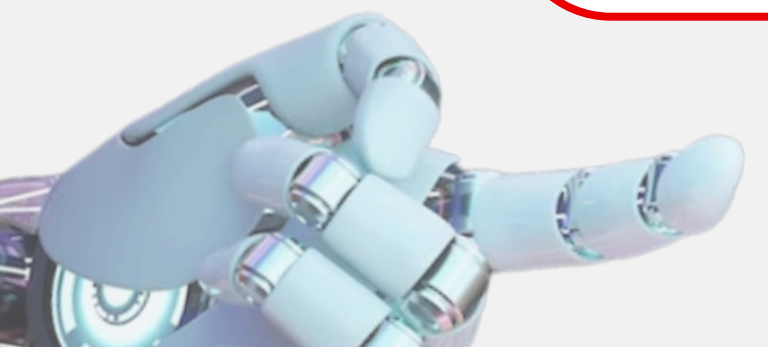
2 UTILIZAR

Checar 100% das bibliotecas recomendadas.

3 UTILIZAR

Treinar times para auditar código gerado.

A responsabilidade pela segurança continua sendo 100% humana.



REFERÊNCIAS

AUSTIN, JACOB ET AL. PROGRAM SYNTHESIS WITH LARGE LANGUAGE MODELS. 15 AGO. 2021.

OLIVEIRA, LAIS APARECIDA FERREIRA DE; OLIVEIRA, CRISTINA CORRÊA DE. O USO DO ASSISTENTE DE PROGRAMAÇÃO BASEADO EM INTELIGÊNCIA ARTIFICIAL, O GITHUB COPILOT, NA QUALIDADE DE SOFTWARE: UMA REVISÃO SISTEMÁTICA DE LITERATURA. RESEARCH, SOCIETY AND DEVELOPMENT, V. 15, N. 4, P. E8615450966, 24 APR. 2026.

PERRY, N.; SRIVASTAVA, M.; KUMAR, D.; BONEH, D. DO USERS WRITE MORE INSECURE CODE WITH AI ASSISTANTS? ACM CCS, 2023. DOI.ORG/10.1145/3576915.3623157 · ARXIV.ORG/ABS/2211.03622

SCHREIBER, MAXIMILIAN; PASCAL TIPPE. SECURITY VULNERABILITIES IN AI-GENERATED CODE: A LARGE-SCALE ANALYSIS OF PUBLIC GITHUB REPOSITORIES. LECTURE NOTES IN COMPUTER SCIENCE, P. 153–172, 19 OUT. 2025.

PEARCE, H. ET AL. ASLEEP AT THE KEYBOARD? ASSESSING THE SECURITY OF GITHUB COPILOT'S CODE CONTRIBUTIONS. IEEE S&P, 2022. DOI.ORG/10.1109/SP46214.2022.9833571 · ARXIV.ORG/ABS/2108.09293

MARK I-CHENG CHEN ET AL. EVALUATING LARGE LANGUAGE MODELS TRAINED ON CODE. ARXIV (CORNELL UNIVERSITY), 7 JUL. 2021.

SPRACKLEN, J. ET AL. WE HAVE A PACKAGE FOR YOU! A COMPREHENSIVE ANALYSIS OF PACKAGE HALLUCINATIONS BY CODE GENERATING LLMS. USENIX SECURITY, 2025. ARXIV.ORG/ABS/2406.10279